

Refactorización de Código Heredado:

Sistema de Tickets de Soporte NLP

Modularización + Tests + Pipeline CI/CD + Documentación

Ejercicio de Clase · Ejercicio 3 de 3 · Proyecto profesional completo con IA como copiloto

Nombre del alumno/a	
Fecha	
 Enlace GitHub del repositorio	

◆ **Instrucciones:** Partirás del código base `app.py` de un sistema NLP para clasificar tickets de soporte. El código funciona pero mezcla responsabilidades, carece de tests y no sigue buenas prácticas. Tu misión es refactorizarlo en módulos dentro de `src/`, escribir tests, configurar un pipeline CI/CD y documentar todo el proceso. Usa la IA como copiloto en cada fase.

BLOQUE 1 — El Código Heredado

A continuación tienes el código base del sistema (`app.py`). Está funcional pero mezcla responsabilidades: modelos de datos, llamadas a la API, lógica de negocio y ejecución directa conviven en un único archivo. Tu trabajo es transformarlo en un proyecto profesional.

```
"""
SISTEMA DE ANÁLISIS DE TICKETS DE SOPORTE CON NLP
=====
OBJETIVOS DEL EJERCICIO:
1. Refactorizar en módulos dentro de src/
2. Crear una clase Pipeline que encadene los pasos
3. Implementar tests unitarios con pytest
4. Configurar pipeline CI/CD con GitHub Actions
"""

import os, json, re
from openai import OpenAI
from dotenv import load_dotenv
from datetime import datetime
from enum import Enum
from typing import Dict, List, Optional

# Cargar variables de entorno ← mezcla config con lógica
load_dotenv()
```

```
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY")) ← cliente global sin inyección

class CategoriaTicket(Enum): ...
class UrgenciaTicket(Enum): ...
class TicketAnalizado: ...

def preprocesar_texto(texto, max_caracteres=10000): ...
def analizar_ticket_con_nlp(texto_usuario): ... ← llama directamente a la API
def transformar_a_ticket(texto_usuario, resultado_api): ...
def ejecutar_accion_segun_ticket(ticket): ... ← lógica de negocio mezclada
def pipeline_procesamiento_ticket(texto_usuario): ...
def procesar_tickets_lote(tickets): ...
def exportar_resultados_json(tickets, archivo): ...

# Ejecución directa mezclada con lógica ← antipatrón
if __name__ == "__main__":
    if not os.getenv("OPENAI_API_KEY"):
        print("ERROR: No se encontró OPENAI_API_KEY")
    else:
        main()
```

1.1 — Identifica los problemas de diseño

Sin refactorizar todavía, lista al menos 5 problemas de diseño del código (no errores de ejecución, sino malas prácticas):

#	Problema detectado	Impacto en producción
1		
2		
3		
4		
5		

BLOQUE 2 — Modularización

Conexión con el módulo anterior: tras analizar el código heredado, el primer paso profesional es separarlo en módulos con responsabilidad única (SRP). El proyecto ya tiene definida la estructura de destino en el README.

2.1 — Diseña la estructura de carpetas

Basándote en el README del proyecto, define qué responsabilidad tiene cada archivo y qué funciones contendrá:

Archivo / Carpeta	Responsabilidad única	Funciones que contiene
src/models.py		
src/preprocess.py		
src/nlp_analyzer.py		
src/actions.py		
src/pipeline.py		
main.py	Punto de entrada	Orquesta y ejecuta

2.2 — Prompt para que la IA refactorice

Redacta el prompt completo que le darías a la IA para que realice la refactorización. Incluye los 5 elementos del prompt profesional (rol, contexto, tarea, restricciones, formato de salida):

Escribe tu prompt aquí:

2.3 — Escribe el código refactorizado

Pega aquí el código de cada módulo tras la refactorización:

```
# src/models.py
# (escribe aquí tu código)
```

```
# src/preprocess.py
# (escribe aquí tu código)
```

```
# src/nlp_analyzer.py
# (escribe aquí tu código)
```

```
# src/actions.py
# (escribe aquí tu código)
```

```
# src/pipeline.py
# (escribe aquí tu código - clase Pipeline que encadene todos los pasos)
```

BLOQUE 3 — Tests Unitarios

Regla profesional: nunca se sube código a producción sin tests. En este bloque escribirás tests para cada módulo usando pytest. El objetivo mínimo es alcanzar un 80% de cobertura de código.

3.1 — Prompt para que la IA genere los tests

Escribe el prompt que usarías para generar los tests unitarios del módulo preprocess.py. Pide: casos felices, casos borde y casos de error:

Escribe tu prompt aquí:

3.2 — Escribe al menos 5 casos de test

Usa la tabla para planificarlos antes de escribir el código:

Nombre del test	Tipo (feliz/borde/erro r)	Input / condición	Resultado esperado
test_1_			
test_2_			
test_3_			
test_4_			
test_5_			

3.3 — Código de los tests

```
# tests/test_preprocess.py
import pytest
from src.preprocess import preprocesar_texto

# Escribe aquí tus tests
```

3.4 — Tests de integración del Pipeline

Crea el archivo tests/test_pipeline.py con al menos 3 tests de integración (usa mocks para la API de OpenAI):

```
# tests/test_pipeline.py
import pytest
```

```
from unittest.mock import patch, MagicMock
from src.pipeline import Pipeline

# Escribe aquí tus tests de integración
```

BLOQUE 4 — Pipeline CI/CD con GitHub Actions

Con el código modularizado y los tests escritos, el último paso es automatizar la validación. Cada vez que hagas *git push*, GitHub Actions ejecutará *linter*, *tests* y *análisis de seguridad* automáticamente.

4.1 — Diseña el fichero `.github/workflows/ci.yml`

Indica qué debe hacer cada fase del pipeline y qué herramienta usarías:

Fase del pipeline	¿Qué debe hacer?	Comando / herramienta
1. Checkout del código		actions/checkout@v4
2. Configurar Python		actions/setup-python@v5
3. Instalar dependencias		pip install -r requirements.txt
4. Ejecutar linter		flake8 / ruff
5. Tests unitarios		pytest
6. Cobertura de tests		pytest --cov
7. Análisis de seguridad		bandit

4.2 — Escribe el fichero `ci.yml` completo

```
# .github/workflows/ci.yml
name: CI Pipeline - Tickets Soporte NLP

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
```

```
runs-on: ubuntu-latest
steps:
  # Completa aquí cada step del pipeline
```

4.3 — ¿Qué ocurre si un test falla en el pipeline?

Explica el flujo: ¿se bloquea el merge? ¿Quién recibe la notificación? ¿Cómo se depura el error desde los logs de GitHub Actions?

Escribe tu respuesta aquí:

BLOQUE 5 — Documentación Profesional

Un proyecto sin documentación es un proyecto muerto. El proyecto ya tiene un README base — tu trabajo es completarlo y añadir documentación técnica en docs/.

5.1 — Estructura de la carpeta docs/

Define qué archivos crearás dentro de docs/ y qué contendrá cada uno:

Archivo en docs/	Contenido / propósito

5.2 — Completa el README.md del proyecto

Complétalo añadiendo: variables de entorno necesarias, cómo ejecutar los tests con cobertura y el badge del pipeline CI:

```
# README.md

# Sistema de Análisis de Tickets de Soporte con NLP

## Descripción

## Instalación
```bash
```

## Variables de entorno

## Uso

## Estructura del proyecto
```
```

## Tests
```bash
```

## Pipeline CI

## Licencia
```

5.3 — Commits semánticos

Indica qué mensaje de commit escribirías para cada cambio. Formato: tipo(scope): descripción corta

| Tipo de cambio | Tu mensaje de commit | ¿Por qué este mensaje? |
|-------------------------|----------------------|------------------------|
| Modularización | | |
| Añadir tests | | |
| CI/CD | | |
| Documentación
README | | |
| Corrección de bug | | |

BLOQUE 6 — Resultado Final del Proyecto

Una vez completada la refactorización, el proyecto debe tener la siguiente estructura. Verifica que tu repositorio coincide antes de entregar:

```

tickets-soporte-nlp/
├─ app.py                # Código heredado original (conservar)
├─ main.py               # Punto de entrada refactorizado
├─ requirements.txt
├─ .env                  # Variables de entorno (no subir a git)
├─ .gitignore
├─ README.md             # Documentación completada
├─ src/                  # ← NUEVA: código modularizado
│   └─ __init__.py
│   └─ models.py          # Enums y clase TicketAnalizado
│   └─ preprocess.py      # Limpieza y normalización de texto
│   └─ nlp_analyzer.py    # Llamada a la API de OpenAI
│   └─ actions.py         # Lógica de acciones por categoría
│   └─ pipeline.py        # Clase Pipeline (orquesta todo)
├─ tests/                # ← NUEVA: tests automatizados
│   └─ __init__.py
│   └─ test_preprocess.py
│   └─ test_nlp_analyzer.py
│   └─ test_actions.py
├─ data/
│   └─ tickets_ejemplo.json
├─ docs/                  # ← NUEVA: documentación técnica
├─ MemoriaFinal/
│   └─ memoriaFinal.md
└─ .github/workflows/ci.yml
  
```

6.1 — Checklist de entrega

| ✓ | Ítem de entrega | Estado |
|--------------------------|--|--------|
| ☐ | app.py original conservado en el repositorio | |
| ☐ | Carpeta src/ con los 5 módulos (models, preprocess, nlp_analyzer, actions, pipeline) | |
| ☐ | Clase Pipeline implementada con logging y manejo de errores | |
| ☐ | Carpeta tests/ con al menos 5 tests unitarios y tests de integración | |
| ☐ | Cobertura de tests $\geq 80\%$ (ejecutar <code>pytest --cov</code>) | |
| ☐ | .github/workflows/ci.yml configurado y funcionando | |
| ☐ | README.md completado con todas las secciones | |
| ☐ | Carpeta docs/ con documentación técnica | |

| | | |
|--------------------------|--|--|
| ☐ | MemoriaFinal.md explicando los problemas del código original | |
| ☐ | Commits semánticos en el historial de git | |

BLOQUE 7 — Reflexión Integrada

Reflexión final: Describe el flujo profesional completo que acabas de implementar. ¿Qué aporta cada paso (análisis → modularización → tests → CI/CD → docs)? ¿Qué hubiera pasado si te saltas alguno en un proyecto real?

7.1 — Flujo profesional completo

Describe con tus palabras qué aprendiste en cada fase y por qué el orden importa:

Escribe tu reflexión aquí:

7.2 — ¿Cómo usaste la IA como copiloto?

Para cada bloque, indica en qué momento pediste ayuda a la IA y cómo validaste su respuesta:

| Bloque | ¿Para qué usé la IA? | ¿Cómo verifiqué su respuesta? |
|---------------------------|----------------------|-------------------------------|
| Bloque 2 — Modularización | | |
| Bloque 3 — Tests | | |
| Bloque 4 — CI/CD | | |
| Bloque 5 — Documentación | | |

7.3 — Autoevaluación

Marca con una X tu nivel de confianza al finalizar el ejercicio:

| Nivel | Descripción | ¿Qué necesitarías repasar? |
|----------------------------|-----------------------|----------------------------|
| ☐ 1 | Necesito más práctica | |

| | | |
|----------------------------|----------------------------|--|
| ☐ 2 | Lo entiendo con ayuda | |
| ☐ 3 | Lo aplico con seguridad | |
| ☐ 4 | Puedo explicárselo a otros | |

BLOQUE 8 — Reflexión MemoriaFinal.md

En la carpeta MemoriaFinal, crea un archivo memoriaFinal.md donde explicarás por qué el código original (app.py) está mal configurado y qué cambios realizasteis.

8.1 — Estructura sugerida para memoriaFinal.md

```
# MemoriaFinal/memoriaFinal.md

# Memoria de Refactorización — Tickets Soporte NLP

## 1. Descripción del código original
<!-- Por qué el app.py original tiene problemas de diseño -->

## 2. Problemas detectados
<!-- Lista numerada de al menos 5 problemas con su impacto -->

## 3. Decisiones de diseño tomadas
<!-- Por qué organizaste los módulos como lo hiciste -->

## 4. Cambios realizados
<!-- Tabla: archivo original → módulo nuevo → razón -->

## 5. Mejoras de calidad conseguidas
<!-- Tests, cobertura, CI/CD, documentación -->

## 6. Lecciones aprendidas
<!-- Reflexión personal sobre el proceso -->
```

8.2 — Borrador de tu memoriaFinal.md



Sistema de Tickets de Soporte NLP



Escribe aquí el contenido completo de tu memoriaFinal.md:

Fecha límite sugerida: 01-Mayo-2026 · Entrega: Carpeta con tu nombre o repositorio en GitHub
Fundación Dicampus · Inclusión Socio-Laboral